



Day 1 – Foundations of Platform Engineering

A hands-on introduction to the principles, practices, and patterns behind world-class Internal Developer Platforms. By the end of today, you'll understand how platform engineering accelerates DevOps, how to design an IDP, and how to sell it internally.

MODULE 1 & 2

FOUNDATIONS TRACK

Today's Agenda

What We'll Cover

01

Module 1: Introduction to Platform Engineering

Why platform engineering exists, key definitions, team composition, value drivers, and maturity models.

03

Module 2: How to Build an IDP – Part 1

Platform as a Product, architecture choices, and the frontend vs. backend split.

02

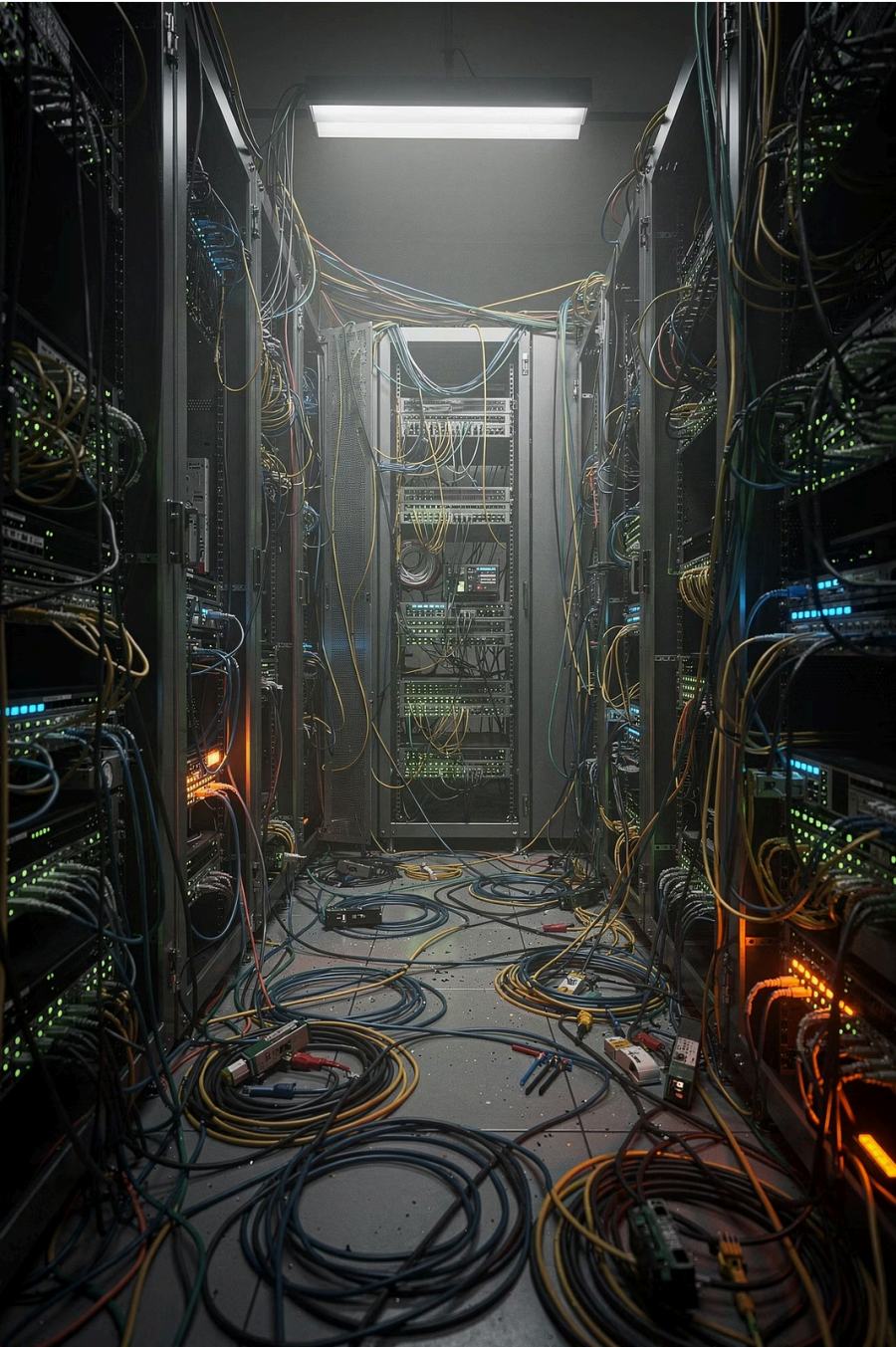
Knowledge Check #1

A quick quiz to reinforce the core concepts from Module 1 before we move forward.

04

Knowledge Check #2

Reinforce your IDP design knowledge with a targeted quiz before we wrap up the day.



The Problem Platform Engineering Solves

Imagine every developer on your team has to set up their own CI/CD pipeline, provision their own cloud infrastructure, configure their own monitoring, and figure out security policies — from scratch, every time. That's not engineering. That's survival. Platform engineering exists to end this chaos.

Module 1

Why Platform Engineering Enables True DevOps

The DevOps Promise

DevOps promised developers would own the full lifecycle — build, deploy, operate. But in practice, developers drowned in ops complexity rather than shipping features.

Platform Engineering as the Fix

Platform engineering delivers the "golden path" — a paved road of curated, self-service tools and workflows. Developers stay focused on product logic. Ops complexity is absorbed by the platform team, not ignored.

Real-world analogy: Spotify's "Backstage" platform gave hundreds of engineers a single, consistent interface to deploy, monitor, and manage services — without filing tickets to infra.

Key Concepts & Definitions

Internal Developer Platform (IDP)

A self-service layer built on top of your infrastructure tooling. Developers interact with it to provision environments, deploy services, and access shared capabilities — without waiting on tickets.

Golden Path

An opinionated, well-supported route through your toolchain. It's not mandatory — but it's the fastest, safest way to get from code to production. Think of it as the "happy path" baked into the platform.

Developer Experience (DevEx)

The sum of all interactions a developer has with tools, processes, and environments. Poor DevEx kills productivity. Great DevEx multiplies it.

Platform Engineering

The discipline of designing, building, and operating IDPs as a product — with real users, roadmaps, feedback loops, and SLAs.

Platform Team Composition

A platform team is not just "infra people." It's a cross-functional product team that serves internal customers — your developers.



Platform Engineers

Build and maintain the core infrastructure, automation, and self-service capabilities.



Product Manager

Owns the IDP roadmap, prioritizes features based on developer feedback, and measures adoption.



Developer Experience Lead

Ensures the platform is intuitive and developer-friendly. May own documentation and onboarding flows.



Security & Compliance

Embeds security policies into the platform so compliance is automatic, not a blocker.

Real-World Example

How Netflix Structures Its Platform Team

Netflix's "Paved Road" team operates like an internal SaaS provider. They publish internal libraries, deployment tools, and observability stacks. Engineering teams are *strongly encouraged* — but not forced — to use them. The result: hundreds of teams deploy independently while maintaining consistency. The platform team tracks adoption metrics like a product company tracks DAU.



Platform Value Drivers

Why does leadership fund a platform team? Because the ROI is measurable. Here are the four core value levers that platform engineering pulls.



Developer Velocity

Reduce time from commit to production. Self-service environments cut provisioning from days to minutes.



Standardization & Compliance

Encode security, observability, and reliability standards into the platform so every service benefits automatically.



Cognitive Load Reduction

Developers focus on business logic, not Kubernetes YAML or IAM policies. The platform absorbs the complexity.



Scalability

One platform team can support hundreds of product teams. Without a platform, every team reinvents the wheel.

The Platform Engineering Maturity Model

Not every organization starts at the same point. Use this model to assess where you are — and plan your next steps.



Level 1 – Ad Hoc

Scripts and wikis. Every team does things differently. No shared platform.



Level 2 – Reactive

Shared tooling exists but is poorly documented. Teams use it inconsistently.



Level 3 – Proactive

A dedicated platform team ships a golden path. Adoption is growing but not universal.



Level 4 – Optimized

IDP is treated as a product. SLAs, roadmaps, user research, and feedback loops are standard practice.

Platform as a Product — The Mindset Shift

Old Mindset: Infra as a Utility

"We provide servers. Developers figure out the rest."

- Ticket-based workflows
- No user research
- Success = uptime
- Developers are "requesters"

New Mindset: Platform as a Product

"Developers are our customers. We build for them."

- Self-service workflows
- Regular developer surveys & interviews
- Success = adoption + satisfaction
- Developers are "users"

 The shift from "infra team" to "platform product team" is cultural, not just technical. It requires empathy for developer pain points and a commitment to roadmap-driven delivery.

Adoption & Internal Selling

Building a great platform means nothing if developers don't use it. Internal adoption requires the same skills as product marketing — you need to communicate value clearly and reduce friction to onboarding.

→ Start with a Champion

Find one developer team willing to pilot the platform. Their success story becomes your case study.

→ Make Onboarding Frictionless

The first 30 minutes of using your platform must be effortless. Write great docs, provide working examples, and offer office hours.

→ Measure and Publicize Wins

Track deploy frequency, provisioning time, and incident reduction. Share these numbers widely with leadership and peers.

→ Iterate Based on Feedback

Run quarterly developer satisfaction surveys (like DORA or DevEx surveys). Act on the results visibly.

Industry Impact & Career Growth

\$10T

Developer Productivity

Estimated value added to the global economy by improving developer productivity by just 10%.

60%

Faster Deployments

Organizations with mature IDPs report up to 60% faster deployment cycles versus those without.

3X

Team Scaling

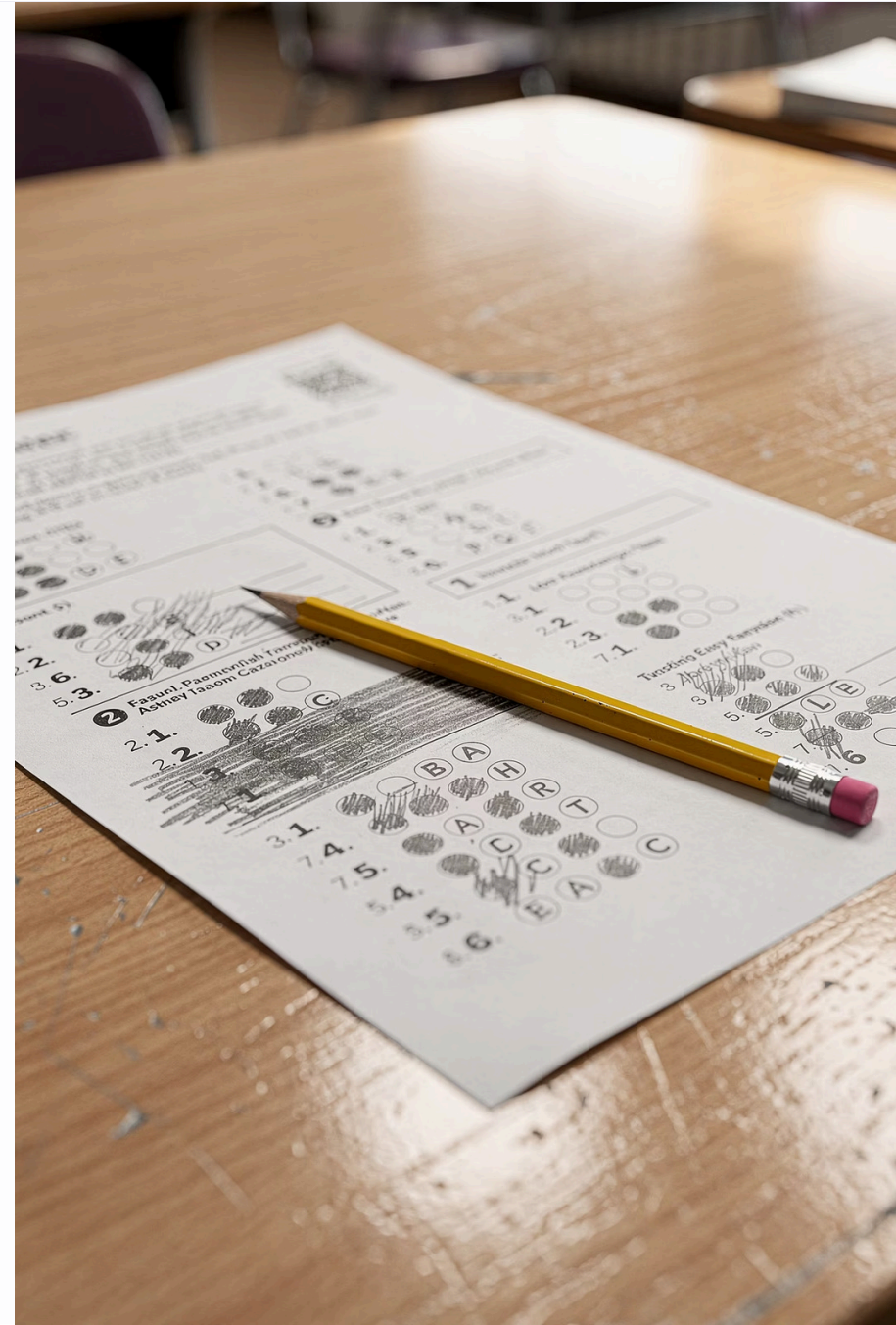
Platform teams enable product engineering to scale 3x faster without proportional infra headcount growth.

Platform engineering is one of the fastest-growing disciplines in software. Roles like Platform Engineer, Developer Experience Engineer, and Internal Tools Product Manager are in high demand across FAANG, scale-ups, and enterprises alike.

Module 1 — Knowledge Check

Quiz Time

Let's verify your understanding of Platform Engineering fundamentals. Answer each question before checking the explanation below.



Quiz — Module 1: Questions

1

What is the "Golden Path"?

- A) A mandatory deployment pipeline B) An opinionated, well-supported route through your toolchain C) A Kubernetes configuration template D) A compliance checklist

2

Which of the following is NOT a platform value driver?

- A) Developer Velocity B) Cognitive Load Reduction C) Vendor Lock-In D) Scalability

3

At Maturity Level 4, what distinguishes a platform team?

- A) They write all the code for product teams B) They treat the IDP as a product with SLAs and roadmaps C) They manage all incident response D) They own the company's CI/CD pipeline exclusively

4

What is the most effective first step for internal platform adoption?

- A) Mandate all teams use the platform immediately B) Start with a champion team and publicize their success C) Remove existing tooling so teams have no alternative D) Hire more platform engineers

Quiz — Module 1: Answers & Explanations

Q1 Answer: B

The Golden Path is an opinionated, well-supported route — not mandatory, but the fastest and safest way to ship. Netflix, Spotify, and Airbnb all have named versions of this concept.

Q2 Answer: C

Vendor Lock-In is a risk, not a value driver. The four value drivers are Velocity, Standardization, Cognitive Load Reduction, and Scalability.

Q3 Answer: B

Level 4 maturity means running the platform like a product company — with user research, SLAs, roadmaps, and feedback loops. It's a cultural and operational shift, not just a technical one.

Q4 Answer: B

Starting with a willing champion team gives you a real-world success story. Mandating adoption without proof of value creates resentment, not adoption.

✔ Great work completing Module 1! Now let's dig into the technical side — building an IDP from the ground up.

MODULE 2

How to Build an IDP — Part 1

Now that we understand *why* platform engineering matters, it's time to get practical. Module 2 covers the architectural decisions, product thinking, and structural choices that define how a real IDP is designed and built.



Platform as a Product — Deep Dive

Treating your IDP as a product means applying everything you know about great software products — to an internal tool. This isn't metaphorical. It's operational.

Define Your Users

Your users are application developers, data engineers, ML engineers, and QA teams. Each persona has different needs. Map them explicitly — don't assume everyone wants the same thing.

Write a Platform Vision

One sentence: "Our IDP enables [user] to [do X] without [pain Y]."
Example: "Our IDP enables backend engineers to deploy to production in under 5 minutes without filing an infra ticket."

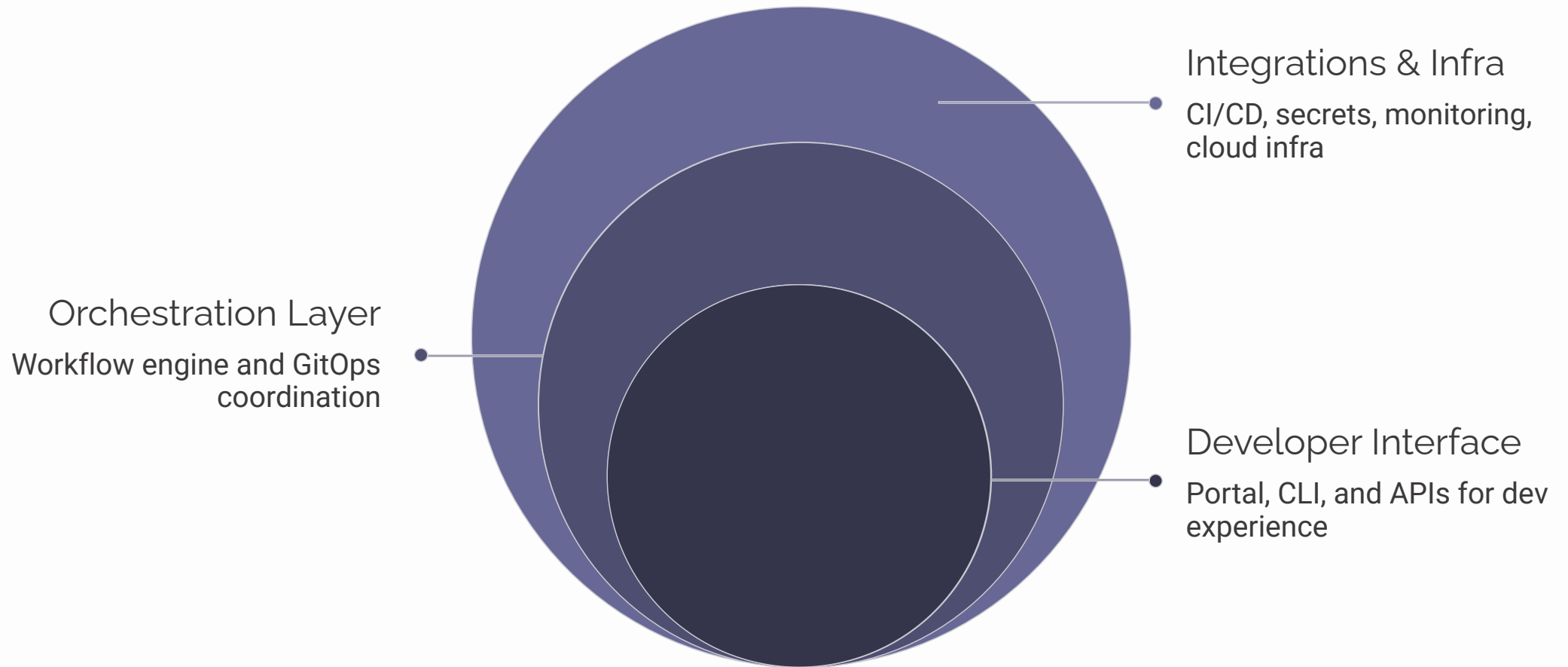
Build a Public Roadmap

Share your quarterly roadmap with all engineering. It builds trust, reduces shadow IT, and invites useful feedback before you build the wrong thing.

Measure Developer Satisfaction

Use DORA metrics, DevEx surveys, or Net Promoter Score (NPS) for internal tools. Track trends quarterly and act visibly on feedback.

IDP Architecture — The Big Picture

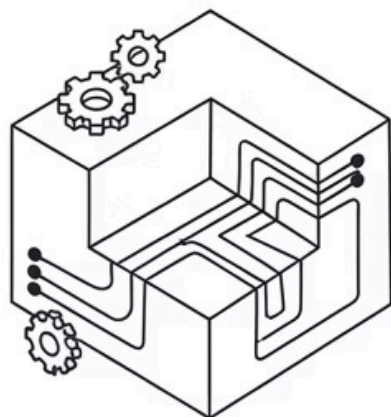


Every IDP — whether built from scratch or assembled from open-source tools — maps to this four-layer model. Understanding which layer you're working in prevents scope creep and helps teams divide ownership cleanly.

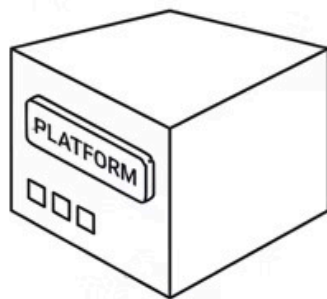
Architecture Choices — Build vs. Buy vs. Assemble

One of the first decisions every platform team faces: how much do we build ourselves? There's no universal answer — but the tradeoffs are well understood.

BUILD FROM SCRATCH (Large Orgs)



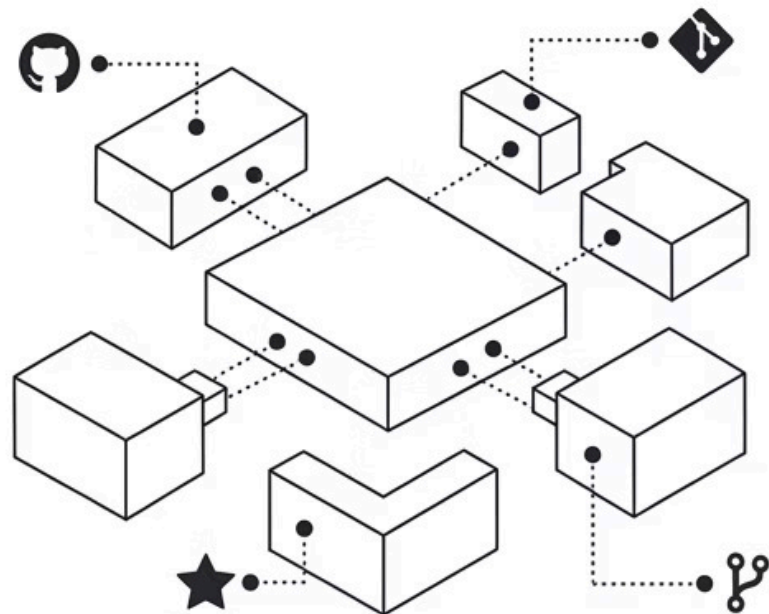
BUY A PLATFORM PRODUCT (SMBs)



**Full Control,
High Cost,
Slow to Deliver**

**Fast to Deploy,
Vendor Dependency,
Limited Customization**

ASSEMBLE OPEN-SOURCE (Engineering-led Orgs like Spotify)



**Flexible, Community-supported,
Requires Integration Effort**



Real-World Example: Spotify's Backstage

What They Built

Spotify created Backstage — an open-source developer portal — to unify their service catalog, documentation, CI/CD pipelines, and infrastructure views into one interface. It's now the most widely adopted IDP framework in the industry, used by American Airlines, Box, and hundreds of others.

Key Lessons

- Start with the service catalog
- Let teams contribute plugins
- Design for discoverability first
- Open-source created a flywheel of contributions

Common Architecture Mistakes to Avoid



Over-Engineering Early

Building a fully custom platform from day one before validating what developers actually need. Start with pain points, not architecture diagrams.



Ignoring Existing Tools

Teams already use GitHub Actions, Terraform, or Helm. Your IDP should wrap and enhance these — not replace them and create migration pain.



Building a Walled Garden

Forcing developers into a rigid platform with no escape hatch drives shadow IT. Build on open standards with extensibility in mind.



No Ownership Model

Platform components with no clear owner go stale fast. Define who maintains each layer, and make it public to build developer trust.

Frontend vs. Backend of an IDP

An IDP has two distinct surfaces: the **frontend** (how developers interact with the platform) and the **backend** (what the platform actually does). Getting this distinction right prevents costly architectural mistakes.

Frontend — The Developer Interface

The surface developers touch daily. Must be intuitive, fast, and self-documenting.

- **Developer Portal** (e.g., Backstage, Port, Cortex)
- **CLI tools** (e.g., internal `platform` CLI)
- **API / GitOps interfaces**
- **Slack bots / chat-ops**

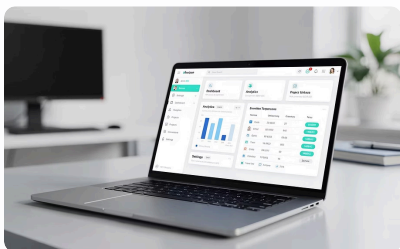
Backend — The Platform Engine

The machinery behind the curtain. Reliability and security are paramount here.

- **Workflow orchestration** (Argo, Temporal)
- **Infrastructure automation** (Terraform, Pulumi)
- **Secret management** (Vault, AWS Secrets Manager)
- **Policy enforcement** (OPA, Kyverno)

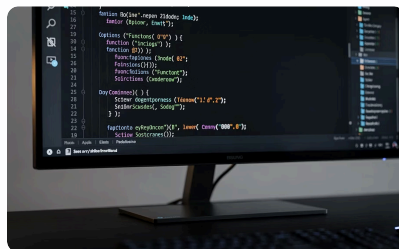
Designing the Developer Portal (Frontend)

The developer portal is the "face" of your platform. It's the first thing developers see, and it shapes their entire perception of whether the platform is worth using.



Service Catalog

A searchable registry of all services, their owners, documentation, and health status. Solves the "who owns this?" problem at scale.



Software Templates

Pre-built scaffolding for new services, repositories, and pipelines. Developers bootstrap a new service in minutes, not days.



Observability Integration

Embed Grafana dashboards, PagerDuty alerts, and deployment history directly in the portal. Everything in one place.

Designing the Platform Backend

The backend is where reliability and security live. Developers may never see it – but they'll feel it immediately if it breaks or if it slows them down.



GitOps as the Control Plane

Use Git as the single source of truth for all platform state. Tools like ArgoCD or Flux sync declared state to running infrastructure automatically.



Abstraction Layers

Expose simplified interfaces to developers (e.g., "create a database") while the backend handles complexity (VPC, IAM, backups, monitoring).



Audit & Compliance Built In

Every action on the platform should be logged and attributable. This isn't optional – it's how you prove security posture to auditors.

Best Practices for IDP Design



Start with Developer Pain, Not Technology

Interview 5–10 developers before writing a line of platform code. The most common complaints (slow onboarding, broken CI, unclear ownership) should dictate your first sprint, not the coolest new tool.



Design for Incremental Adoption

Modular IDPs win. Let teams adopt one piece at a time (e.g., just the service catalog, or just the deployment templates). Don't require a "big bang" migration.



Document as a First-Class Citizen

Treat docs like code. Use tools like TechDocs (built into Backstage) to keep documentation co-located with the service it describes. Stale docs kill trust faster than bugs do.

Applying the Concepts

A Day in the Life of a Developer with a Great IDP



Frontend vs. Backend — Choosing Your Tools

Here's a practical tool landscape to help orient architecture decisions. This is not exhaustive — it's a starting point for conversation.

Layer	Popular Tools	Key Consideration
Developer Portal	Backstage, Port, Cortex, OpsLevel	Extensibility via plugins; don't build custom if OSS fits
CI/CD	GitHub Actions, Tekton, ArgoCD, CircleCI	Standardize on one; support escape hatches
Infrastructure as Code	Terraform, Pulumi, CDK	Use modules/abstraction to hide complexity from devs
Secrets Management	HashiCorp Vault, AWS Secrets Manager	Centralize; never let teams manage secrets ad hoc
Observability	Grafana, Datadog, OpenTelemetry	Embed in portal; make dashboards zero-config for new services

Avoiding Common IDP Mistakes — Recap

Don't: Build Everything at Once

An IDP that tries to solve every problem immediately ships nothing useful for months. Prioritize ruthlessly.

Don't: Skip Developer Research

Building without user feedback produces a platform nobody uses. Talk to developers weekly during early development.

Do: Ship an MVP in 6–8 Weeks

A working service catalog with basic templates is more valuable than a perfect platform 12 months away. Ship, learn, iterate.

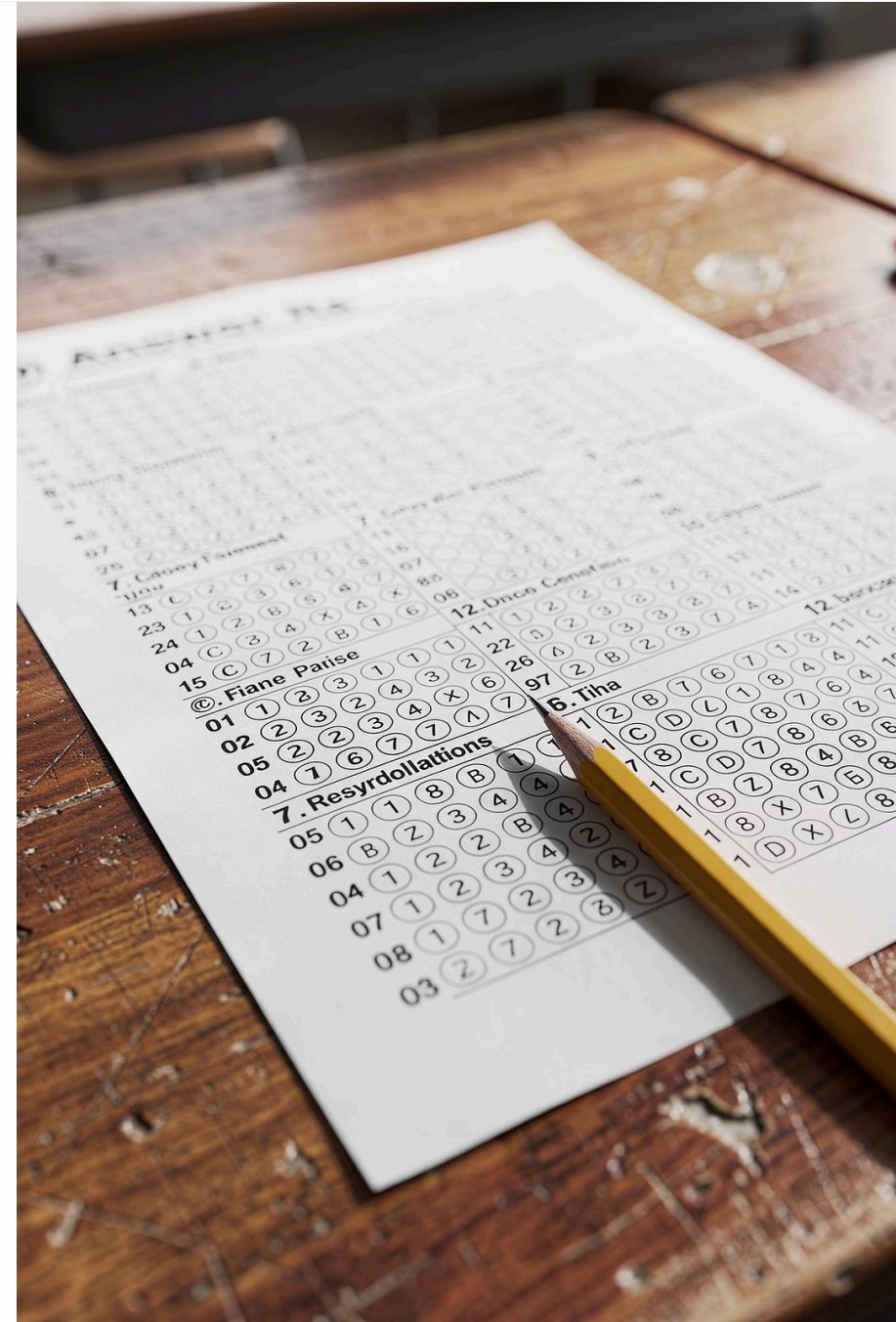
Do: Celebrate Early Adopters

Recognize teams who adopt early and share their results. Social proof is your most powerful adoption tool.

Module 2 — Knowledge Check

Quiz Time

Test your understanding of IDP design, architecture choices, and the frontend vs. backend split. Think carefully before checking the answers!



Quiz — Module 2: Questions

1

Which of the following is a developer portal (IDP frontend) tool?

A) Terraform B) HashiCorp Vault C) Backstage D) ArgoCD

2

What does "GitOps as the Control Plane" mean?

A) All code must be reviewed by the platform team B) Git is the single source of truth for platform state, synced automatically C) GitHub Actions is the only allowed CI tool D) Developers push directly to production via Git

3

What is the recommended first step when building an IDP?

A) Choose a cloud provider B) Design a microservices architecture C) Interview developers to identify their biggest pain points D) Set up a Kubernetes cluster

4

What is the key advantage of an "assemble open-source" IDP approach?

A) Zero integration effort B) Flexibility with community support C) No engineering skill required D) Guaranteed vendor SLA

Quiz — Module 2: Answers & Explanations

Q1 Answer: C — Backstage

Backstage is an open-source developer portal created by Spotify. Terraform is IaC (backend), Vault is secrets management (backend), and ArgoCD is a GitOps CD tool (backend).

Q2 Answer: B

GitOps means Git is the single source of truth. Tools like ArgoCD or Flux watch your Git repo and automatically reconcile running infrastructure to match the declared state.

Q3 Answer: C

Always start with developer research. The most common trap is building a technically impressive platform that solves the wrong problems. Talk to your users first.

Q4 Answer: B

Assembling open-source tools (like Backstage + ArgoCD + Vault) gives you flexibility and a strong community ecosystem, but requires integration effort and in-house expertise to maintain.

✅ Excellent work completing both modules! You've covered the foundations of platform engineering and the essentials of IDP design. See you in Day 2!

Day 1 — Key Takeaways



Platform Engineering Enables Real DevOps

It removes the operational burden from developers by providing a curated, self-service platform — turning DevOps from a philosophy into a daily reality.



Frontend + Backend = Complete Platform

Great IDPs invest in both the developer interface (portal, CLI) and the platform engine (GitOps, IaC, secrets, policies). Neither layer can succeed without the other.



Treat Your IDP as a Product

User research, roadmaps, adoption metrics, and feedback loops are not optional extras. They are what separates a thriving platform from an abandoned one.



Start Small, Ship Fast, Iterate

The best platform teams ship an MVP in 6–8 weeks, learn from real users, and build from there. Perfectionism is the enemy of adoption.